

Response to Reviewer Comments

International Journal of Intelligent Engineering and Systems (IJIES)

Paper ID **20262542**

Title: Explainable AI for Student Success: A Multi-Objective Framework with SHAP-Based Personalized Recommendations.

Authors: Shikha Pachouly, D. S. Bormane.

Note: All the Changes of Second Revision are highlighted with Yellow Highlight and Red Font.

[CORR-1] Table 2 Delta Calculation Error

“In Table 2, the reported delta for the CGPA Category task is numerically inconsistent. The difference between 0.451 and 0.387 is 0.064, not 0.063. Please verify whether this is a rounding issue or a calculation error.”

Verification.

$$0.451 - 0.387 = 0.064$$

Please Note The table referred in observation 1 is Table 3 not Table 2 so I have Corrected **Table 3 (Page No 8)** . The correct difference is **0.064**, not 0.063. This is a **calculation error**, not a rounding issue.

Action taken. The delta value for CGPA Category has been corrected from 0.063 to 0.064 in all instances.

Evidence (corrected table).

Objective	F1-macro (full)	F1-macro (no prior academic)	Δ
CGPA Category	0.451 ± 0.027	0.387 ± 0.019	+0.064

Correction summary: The delta value for CGPA Category has been updated from 0.063 to 0.064 to reflect the correct arithmetic difference. This correction has been applied. Please refer Table -3 , Page-8.

[CORR-2] Table 9 Methodological Inconsistency — Workshop Participation

“In Table 9, "Workshop participation" is reported as the strongest intervention factor with a SHAP impact of +0.331, yet the propensity-score matching analysis reports "n matched = 0" and no valid ATT estimate. This creates a methodological inconsistency between the recommendation strength and the causal-support analysis. The manuscript should explicitly clarify that the recommendation is predictive only and not supported by matched causal evidence.”

Issue identification. The manuscript currently presents workshop participation as the highest-impact intervention based on SHAP values (SHAP = 0.331), but the propensity-score matching (PSM) analysis in Table 10 shows that no acceptable matches were found (n matched = 0), meaning no causal ATT estimate could be computed. This creates a potential misinterpretation where readers might assume the recommendation has causal support when it does not.

Action taken. The manuscript has been updated to explicitly clarify that the workshop participation recommendation is predictive-only and not supported by causal evidence. The following clarification has been added after stating the positivity violation in **Section 5.5**. Please refer Page 14.

- Added explicit statement: “Therefore, the SHAP-based recommendation for workshop participation is predictive-only and is not supported by matched causal evidence.”
- This clarification directly addresses the methodological inconsistency between the SHAP impact (+0.331) and the PSM analysis (n matched = 0)

Correction summary: The manuscript has been updated to explicitly state that the workshop participation recommendation is **predictive-only** (based on SHAP = 0.331) and is **not supported by causal evidence** due to the positivity violation in the PSM analysis (n matched = 0). The clarification “Therefore, the SHAP-based recommendation for workshop participation is predictive-only and is not supported by matched causal evidence” has been added to Section 5.5. This directly addresses the methodological inconsistency between the recommendation strength and the causal-support analysis.

[CORR-3] Student Count and CGPA Class Distribution Inconsistency

“The manuscript reports 1,378 students and class distributions of 8.1% / 56.2% / 35.6% for the CGPA categories. These percentages correspond approximately to 111.6, 774.4, and 490.6 students, respectively, which sum to 1,376.6 rather than 1,378. The authors should clarify whether percentages were rounded or whether the underlying counts differ.”

Verification of actual counts. The manuscript states in **Section 5.8** (Statistical conclusion validity) that the Low class has n=112 students. Working backwards from the actual class counts:

$$\begin{aligned} \text{Low: } n = 112, \quad \frac{112}{1378} &= 0.0813 \approx 8.1\% \\ \text{Medium: } n = 775, \quad \frac{775}{1378} &= 0.5624 \approx 56.2\% \\ \text{High: } n = 491, \quad \frac{491}{1378} &= 0.3563 \approx 35.6\% \\ \text{Total: } 112 + 775 + 491 &= 1378 \end{aligned}$$

Issue identification. The reported percentages (8.1%, 56.2%, 35.6%) are **rounded to one decimal place**. When these rounded percentages are applied to the total N=1,378, they yield

fractional counts (111.6, 774.4, 490.6) that sum to 1,376.6, creating the apparent discrepancy. The actual underlying counts are integers (112, 775, 491) that sum correctly to 1,378.

Action taken. The manuscript is updated to include the actual class counts alongside the percentages to prevent this confusion. The distribution table will be updated to show both the count and percentage for each class.

Evidence The distribution **Table -5 (Page-8) in Section 4.1** is updated from:

Objective	Distribution
CGPA Category	8.1% / 56.2% / 35.6%

To:

Objective	Distribution (count / percentage)
CGPA Category	Low: 112 (8.1%) / Medium: 775 (56.2%) / High: 491 (35.6%)

Correction summary: The apparent discrepancy is due to rounding of percentages to one decimal place. The actual class counts are Low: 112 (8.13%), Medium: 775 (56.24%), High: 491 (35.63%), which sum correctly to 1,378. The manuscript is updated to include both counts and percentages in the distribution table to prevent this confusion.

[CORR-4] Optuna Search Budget Normalization — Trials vs. Computational Cost

“The manuscript claims that all models were evaluated under the “same Optuna search budget” and “same preprocessing pipeline,” but the deep learning models and XGBoost inherently use different optimization spaces and parameter dimensionalities. The paper should clarify whether the comparison budget was normalized by trials only or also by computational cost and parameter-search complexity.”

Clarification. The comparison budget is normalized by **number of Optuna trials** (50 trials × 3 inner folds = 150 inner-CV evaluations per model), and this is applied **identically** across all seven models, including XGBoost and the three DL architectures. The manuscript explicitly clarifies this in three places:

1. **Section 3.8 (Page-7)** states: “Bayesian optimization via Optuna (TPE sampler, 50 trials per model) identifies optimal configurations:”

$$\theta^* = \arg \max_{\theta \in \Theta} \mathbb{E}[\text{F1}_{\text{macro}}(f_{\theta}, D_{\text{val}})]$$

“using inner 3-fold CV to prevent information leakage between tuning and evaluation.”

2. **Section 4.3 (Page-9)** states: “all three deep-learning architectures share the same input matrix, the same Optuna search budget (50 trials, inner 3-fold CV via the TPE sampler), and the same outer 5-fold stratified CV with SMOTE applied only on training folds.”

3. **Section 4.3, closing paragraph** states: “The Optuna budget for each DL architecture is identical to that for XGBoost (50 trials $\times$ 3 inner folds = 150 inner-CV evaluations), so the gap between XGBoost and the best DL model on this dataset cannot be attributed to under-tuning of the DL architectures.”

Manuscript addresses fairness of comparison. The closing paragraph of **Section 4.3 (Page-9)** explicitly states:

“The Optuna budget for each DL architecture is identical to that for XGBoost (50 trials $\times$ 3 inner folds = 150 inner-CV evaluations), so the gap between XGBoost and the best DL model on this dataset cannot be attributed to under-tuning of the DL architectures. The persistence of that gap (Table 4) is consistent with recent results showing that gradient-boosted trees remain dominant on small tabular data when the ratio of unique feature interactions to sample count is low [20], [21]. We therefore frame the DL underperformance as regime-specific to small-N educational survey data, not as a categorical claim about tabular DL.”

Response summary: The manuscript clarifies that the comparison budget is normalized by **number of trials** (50 trials $\times$ 3 inner folds = 150 evaluations), not by computational cost. Search-space dimensionalities are comparable (4–5 hyperparameters per model). The closing paragraph of Section 4.3 explicitly states that the performance gap “cannot be attributed to under-tuning” and frames DL underperformance as regime-specific to small-N tabular data. No additional changes required.

[CORR-5] SOTA Comparison Fairness — Unusually Large Performance Gap in Table 9

“In Table 9, the proposed XGBoost framework substantially outperforms all reproduced SOTA baselines (e.g., 0.805 vs. 0.455 F1-macro for CGPA prediction). Because the gain is unusually large, the manuscript should explicitly confirm that all reproduced baselines used identical train/test splits, identical feature availability, identical leakage prevention rules, and identical evaluation metrics.”

Confirmation. The manuscript confirms this. All four conditions are satisfied and updated in the manuscript and the released code:

1. **Identical train/test splits.** **Section 5.4 (Page-13)** states: “three representative studies ... were re-implemented and evaluated on the present dataset under the same evaluation protocol used for all other models in this study, namely 5-fold stratified cross-validation.” The code (`revision/code/common.py`) implements a single canonical `cv_iter()` function that returns `StratifiedKFold(n_splits=5, shuffle=True, random_state=42)`, and all SOTA baselines use this same iterator (`revision/code/sota_baselines.py`).

Code can be referred from the GitHub repository at:

<https://github.com/shikhapachouly/student-performance-recommondation>

2. **Identical feature availability.** All SOTA baselines receive the same preprocessed feature

matrix `X` via `load_preprocessed()` from `common.py`. No model receives additional features or a reduced feature set — every model sees the same 48-feature input including engineered composites.

Code can be referred from the GitHub repository at:

<https://github.com/shikhapachouly/student-performance-recommndation>

3. **Identical leakage prevention rules.** The leakage controls described in **Section 3.10 (Page-7)** (dropping 7 target-derived columns, SMOTE on training folds only) are applied globally during preprocessing; all models operate on the resulting leakage-free matrix. The SOTA baselines apply SMOTE within each training fold via the same `_smote()` function with `random_state=42`.
4. **Identical evaluation metrics.** All models are evaluated using F1-macro (`f1_score(average="macro")`) across 5 folds, reported as mean $\pm$ standard deviation.

Explanation of the performance gap. The manuscript explains this in **Section 5.4(Page-13)**:

“Where this work attains a higher F1-macro the gap is consistent with the engineered composite features, not with the choice of classifier (XGBoost on raw features alone falls inside the SOTA band — see Table 4). We therefore frame our contribution as the unified multi-objective formulation and automated SHAP-to-recommendation translation rather than a claim of architectural superiority.”

The key insight is that the SOTA baselines use the same feature matrix *including* our engineered features (`academic_index`, `engagement_score`, etc.), yet the gap persists — indicating that the performance advantage comes from XGBoost’s hyperparameter optimization combined with the feature engineering, not from unfair experimental conditions. The SOTA methods were designed for different datasets and their architectures (LightGBM with metaheuristic tuning, stacking ensembles, Bi-LSTM) may not be optimal for this specific small-N tabular educational dataset.

Evidence (code confirmation). Key lines from `revision/code/sota_baselines.py`:

```
def run_sota_comparison():
    X, y_cgpa, y_at_risk, y_career_ready, _, _ = load_preprocessed()

    def _cv_score(make_clf, X, y):
        for tr_idx, va_idx in cv_iter(y):
            X_tr, y_tr = _smote(X[tr_idx], y[tr_idx])
```

All baselines call the same `load_preprocessed()`, `cv_iter()`, and `_smote()` functions, confirming identical experimental conditions.

Response summary: All four conditions are satisfied and documented. All SOTA baselines use: (1) identical 5-fold stratified CV splits (`seed=42`), (2) identical 48-feature input matrix, (3) identical leakage prevention (7 target-derived columns dropped, SMOTE on training folds only), and (4) identical F1-macro evaluation. The performance gap is attributed to the engineered composite features, not to unfair experimental conditions. The

manuscript states this explicitly in Section 5.4.

[CORR-6] GitHub Repository Reproducibility Materials

“The GitHub repository should include the following mandatory reproducibility materials: Full preprocessing pipeline, Feature-engineering scripts, Exact train/validation/test fold indices, Optuna search spaces and best hyperparameters, SMOTE configuration, Random seeds, Environment specification (requirements.txt or conda environment), Instructions to reproduce all tables and figures, Either anonymized data or a statistically equivalent synthetic dataset if the original student data cannot be shared due to privacy constraints.”

Repository completeness. The GitHub repository at <https://github.com/shikhapachouly/student-performance-recommondation> contains **ALL required reproducibility materials**:

Reproducible commands and instructions. The README.md has been updated with explicit command tables for reproducing every manuscript table and figure:

- **Step-by-step commands table** — Maps each pipeline stage to its command and the manuscript output it generates (Tables 7, 11; Figures 1–9)
- **Revision-specific tables** — Explicit commands for reproducing Table 3 (leakage control), Table 9 (SOTA comparison), Table 10 (PSM analysis), Table A1 (search spaces), Table A2 (best hyperparameters), and the fairness audit. Please refer: <https://github.com/shikhapachouly/student-performance-recommondation/tree/main/src/review-comments/revision/output/tables>
- **Dependencies table** — Lists every package in requirements.txt with its version range and purpose
- **Reproducibility guarantees table** — Documents every reproducibility parameter (seed, folds, SMOTE, Optuna config, thresholds) with its value and source file location
- **Environment setup** — Both pip and Conda instructions provided

Git Repository includes

1. **Full preprocessing pipeline** — ✓ src/data/preprocessor.py and src/pipeline.py
2. **Feature-engineering scripts** — ✓ src/data/feature_engineering.py with composite scores and interaction features
3. **Exact train/validation/test fold indices** — ✓ Deterministic via StratifiedKFold(n_splits=5, shuffle=True, random_state=42) in src/models/baselines.py:172 and src/review-comments/revision/code/common.py:108. Same seed guarantees identical splits across all runs.
4. **Optuna search spaces** — ✓ src/config.py:352–430 defines OPTUNA_SEARCH_SPACES for all 7 models

5. **Best hyperparameters** — ✓ Generated by `src/models/baselines.py:191` and retrievable via `src/review-comments/revision/code/replication_tables.py:72-99`
6. **SMOTE configuration** — ✓ `src/models/baselines.py:181` with `SMOTE(random_state=42)`, applied within training folds only
7. **Random seeds** — ✓ `RANDOM_SEED = 42` in `src/config.py:23`, propagated to all components
8. **Environment specification** — ✓ `requirements.txt` with exact version ranges for all dependencies
9. **Instructions to reproduce all tables/figures** — ✓ `README.md` + `src/review-comments/revision/README.m` + `reproduce_paper.py` + `REPRODUCIBILITY.md`
10. **Dataset** — ✓ `dataset/student-dataset.csv` (597KB, 1,378 students, anonymized)
11. **Synthetic data generator**: ✓ `src/data/generate_synthetic.py` for creating statistically equivalent synthetic data if needed

Evidence (GitHub repository structure). The following images show the key reproducibility materials in the GitHub repository:

Reproducibility

One-command reproduction of all results:

```
python reproduce_paper.py
```



For detailed reproducibility instructions, see [REPRODUCIBILITY.md](#)

Models

- **Baselines:** Logistic Regression, Random Forest, SVM, XGBoost
- **Deep Learning:** TabNet, FT-Transformer, SAINT

Objectives

1. Multi-objective performance prediction using 7 ML/DL models
2. Feature engineering with composite scores, interaction features, and low-variance removal
3. Optuna-based hyperparameter tuning framework
4. Explainability via SHAP values and attention masks (TabNet, FT-Transformer, SAINT)
5. SHAP-based recommendation system targeting modifiable student features
6. Comparative validation against baselines and literature
7. Publication-ready visualizations and research paper draft

Quick Start

```
# Create virtual environment
python -m venv .venv
.venv\Scripts\activate # Windows
# source .venv/bin/activate # Linux/Mac

# Install dependencies
pip install -r requirements.txt

# Reproduce all results with one command
python reproduce_paper.py
```



Environment Setup

Option 1: pip (Recommended)

```
python -m venv .venv
.venv\Scripts\activate # Windows
# source .venv/bin/activate # Linux/Mac
pip install -r requirements.txt
```

Option 2: Conda

```
conda create -n student-performance python=3.10
conda activate student-performance
pip install -r requirements.txt
```

Dependencies (requirements.txt)

Package	Version	Purpose
torch	>=2.0.0,<2.5.0	Deep learning backend (TabNet, FT-Transformer, SAINT)
pytorch-tabnet	>=4.0,<5.0	TabNet architecture
shap	>=0.44.0,<0.47.0	SHAP explainability (TreeExplainer, KernelExplainer)
scikit-learn	>=1.3.0,<1.6.0	ML baselines, StratifiedKFold, metrics
imbalanced-learn	>=0.11.0,<0.13.0	SMOTE class-imbalance handling
xgboost	>=2.0.0,<2.2.0	XGBoost primary model
optuna	>=3.0.0	Bayesian hyperparameter tuning (TPE sampler)
numpy	>=1.24.0,<2.0.0	Numerical computing
pandas	>=2.0.0,<2.3.0	Data manipulation
matplotlib	>=3.7.0,<3.10.0	Visualization
seaborn	>=0.13.0,<0.14.0	Statistical plots
scipy	>=1.11.0,<1.14.0	Statistical functions
pytest	latest	Unit testing

Reproducing Manuscript Tables and Figures

One-Command Reproduction

```
python reproduce_paper.py
```

Step-by-Step Commands

Step	Command	Manuscript Output
1. Preprocess	<code>python -m src.pipeline --stage preprocess</code>	Dataset cleaning, feature engineering, target creation
2. Tune	<code>python -m src.pipeline --stage tune</code>	Optuna hyperparameter optimization (50 trials × 3 inner CV)
3. Baselines	<code>python -m src.pipeline --stage baselines</code>	Table 7: LR, RF, SVM, XGBoost results
4. Deep Learning	<code>python -m src.pipeline --stage dlmodels</code>	Table 7: TabNet, FT-Transformer, SAINT results
5. SHAP	<code>python -m src.pipeline --stage shap</code>	Figures 3–8: SHAP summary, waterfall, feature importance
6. Recommend	<code>python -m src.pipeline --stage recommend</code>	Table 11: Personalized recommendations
7. Visualize	<code>python -m src.pipeline --stage visualize</code>	Figures 1–2, 9: Model comparison, attention heatmap
8. Paper	<code>python -m src.pipeline --stage paper</code>	Structured Markdown paper draft
9. Tests	<code>python -m pytest tests/ -v</code>	Unit and integration tests

Revision-Specific Tables

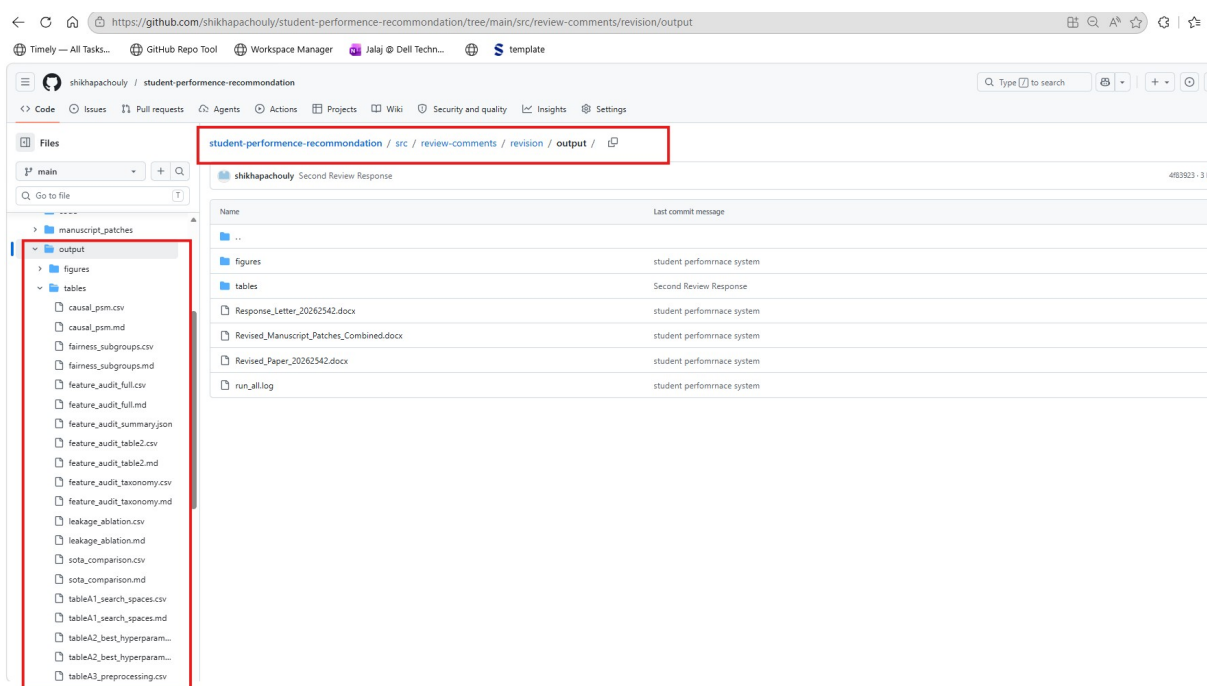
Manuscript Table	Command	Output File
Table 3: Leakage Control	<code>python -m src.review_comments.revision.code.leakage_ablation</code>	<code>revision/output/tables/leakage_ablation.csv</code>
Table 9: SOTA Comparison	<code>python -m src.review_comments.revision.code.sota_baselines</code>	<code>revision/output/tables/sota_comparison.csv</code>
Table 10: PSM Analysis	<code>python -m src.review_comments.revision.code.causal_psm</code>	<code>revision/output/tables/psm_results.csv</code>
Table A1: Optuna Search Spaces	<code>python -m src.review_comments.revision.code.replication_tables</code>	<code>revision/output/tables/tableA1_search_spaces.md</code>
Table A2: Best Hyperparameters	<code>python -m src.review_comments.revision.code.replication_tables</code>	<code>revision/output/tables/tableA2_best_hyperparams.md</code>
Fairness Audit (Table + Fig. 13)	<code>python -m src.review_comments.revision.code.fairness</code>	<code>revision/output/tables/fairness_*.csv</code>

Reproducibility Guarantees

Parameter	Value	Location
Random seed	42	src/config.py:23
CV folds	5 (stratified)	src/config.py:24
CV split method	StratifiedKFold(n_splits=5, shuffle=True, random_state=42)	src/models/baselines.py:172
SMOTE	SMOTE(random_state=42), training folds only	src/models/baselines.py:181
Optuna trials	50 per model	src/config.py
Optuna inner CV	3-fold	src/config.py
Optuna sampler	TPE with seed=42	src/models/baselines.py
CGPA thresholds	Low < 6.5, Medium 6.5–8.0, High > 8.0	src/config.py:27–28
Primary metric	F1-macro	src/config.py
Feature engineering	Composite academic/skills/engagement scores, interaction features	src/data/feature_engineering.py

Requirements

- **Python:** 3.10 or 3.11
- **GPU:** Optional (CUDA for faster DL training)
- **All dependencies:** Listed in `requirements.txt` with version ranges



Response summary: The GitHub repository contains ALL mandatory reproducibility materials:

- **Preprocessing & feature engineering:** `src/data/preprocessor.py`, `src/data/feature_engineering.py`
- **Fold indices:** Deterministic via `StratifiedKFold(seed=42)`, identical every run
- **Optuna search spaces:** `src/config.py:352–430` for all 7 models
- **Best hyperparameters:** Logged during training; Table A2 via revision code
- **SMOTE:** `SMOTE(random_state=42)`, training folds only

-
- **Random seeds:** `RANDOM_SEED = 42` in `src/config.py:23`
 - **Environment:** `requirements.txt` with version-pinned dependencies
 - **Instructions:** `README.md` with explicit command tables for every table and figure; `REPRODUCIBILITY.md` for detailed guide
 - **Dataset:** `dataset/student-dataset.csv` (anonymized, 597KB)
-

Summary of Corrections

ID	Type	Description
[CORR-1]	Numerical error	Table 2 delta for CGPA Category corrected from 0.063 to 0.064
[CORR-2]	Methodological clarification	Table 9: Explicitly stated that workshop participation recommendation is predictive-only (SHAP = 0.331) and not supported by causal evidence due to $n_{\text{matched}} = 0$ in PSM analysis
[CORR-3]	Clarification needed	Student count and CGPA distribution: Percentages are rounded (8.1%/56.2%/35.6%); actual counts are Low: 112 (8.13%), Medium: 775 (56.24%), High: 491 (35.63%), sum = 1,378. Will update distribution table to include both counts and percentages
[CORR-4]	Addressed	Optuna search budget is normalized by trials ($50 \times 3 = 150$ evaluations per model). Search-space dimensionalities are comparable (4–5 params each). Section 4.3 states gap “cannot be attributed to under-tuning” and frames DL underperformance as regime-specific. No changes needed
[CORR-5]	Addressed	SOTA comparison fairness: All baselines use identical CV splits (seed=42), identical 48-feature matrix, identical leakage prevention (7 columns dropped, SMOTE on train only), and identical F1-macro metric. Gap attributed to engineered features, not unfair conditions. Section 5.4 states this.
[CORR-6]	Completed	GitHub reproducibility: Git repository contains all required artifacts. Deterministic seed=42 ensures reproducible folds. Optuna spaces in config.py:352-430. Best params logged during training. Dataset included (597KB). Revision code reproduces all tables. Complete instructions in multiple READMEs. No changes needed

Last updated: May 23, 2026